MODULE 31 · DOMAIN 7 - DEVELOPER PRODUCTIVITY AND ENABLEMENT

# AI-Assisted Workflows and Operational Troubleshooting

Claude in the workflow rather than a side channel, pull request review with human approval, grounded documentation, read-only production data, CI runs, the rollout sequence, and debugging from hypothesis to fix.

DOMAIN 7 of 7 EXAM WEIGHT 7% MODULE 31 of 38

admin@sthithapragna.com

One mechanic swaps parts until the noise stops. Another makes the noise happen on demand first. **Only one of them knows what they fixed.**

# In the workflow, not a side channel

IN THE WORKFLOW

## IDE, terminal, CI

- Sees the repository and its conventions
- Changes arrive as diffs you review
- Runs tests and linters itself
- Governed by managed and project settings
- Leaves a review trail in pull requests

A SIDE CHANNEL

## Copy and paste into a chat

- No repository context
- Code pasted out, answers pasted back
- Nothing runs, nothing is tested
- Outside organisational controls
- No record of what changed or why

THE TIEBREAKER

Put Claude where the work and the controls already are: the IDE, the terminal and the pipeline.

# Pull request review: Claude comments, humans approve

01
### Open
A developer opens a pull request.

02
### Review
Claude runs in CI with the team's checklist.

03
### Comment
Findings appear inline on the diff.

04
### Respond
The author fixes or explains.

05
### Approve
A human reviewer approves.

06
### Merge
Branch protection requires that approval.

ON THE EXAM

Claude adds findings. A person owns the approval. Branch protection makes that a rule, not a habit.

# Documentation grounded in the code

- **Generated from the source of truth:** code, contracts, ADRs, configuration.
- **Delivered as a pull request,** reviewed by the owning team like any change.
- **Stored with the code,** so it changes when the code changes.
- **Flagged for refresh** when the code it describes changes.
- **Never published unreviewed,** however fluent it reads.

> ON THE EXAM
>
> Fluent is not accurate. Generated documentation goes through the same review as code.

# Production data: read-only by default

- **Connect with a read-only database user,** so queries cannot modify data.
- **Writes through separate, named tools** with ask rules, so a person confirms each one.
- **Least data:** masked or sampled where the task allows.
- **Every query and write logged** with the user who triggered it.
- **Never** admin credentials with auto approval.

> ON THE EXAM
>
> Reading is the default. Writing is an exception, confirmed by a person, every time.

# Running Claude Code headless in CI

| CONTROL | WHY |
| --- | --- |
| -p, non-interactive | No person is there to answer prompts |
| --output-format json | Results the pipeline can parse |
| --allowedTools, narrow | Pre-approve only what the job needs |
| dontAsk mode | Anything not pre-approved is denied |
| --max-turns | Bounds a runaway loop |
| disabledMcpjsonServers | -p runs load project MCP servers without asking |

THE TIEBREAKER

In CI nobody answers prompts. Decide every permission in advance, and deny the rest.

# The rollout sequence

01
**Value**
Where it helps; who keeps authority.

02
**Baseline**
Managed and project settings, CLAUDE.md, MCP.

03
**Pilot**
One or two teams; measure.

04
**Iterate**
Fix the configuration from findings.

05
**Roll out**
In waves, with training.

06
**Monitor**
Telemetry and outcomes.

ON THE EXAM

Value and authority first, configuration second, people third. A big-bang rollout spreads mistakes.

# Measuring adoption honestly

- **Telemetry:** OpenTelemetry metrics for sessions, tokens, cost and lines of code.
- **Outcomes:** cycle time, review time, escaped defects, incidents.
- **Baselines** from before the rollout, per team.
- **Not lines of code:** more code is not more value.
- **Ask the developers:** where it helps, where it gets in the way.

> ON THE EXAM
>
> Usage shows adoption. Outcomes show value. Report both, against a baseline.

# Debugging: hypothesis before fix

01
### Observe
Symptoms, scope, when it started.

02
### Hypothesise
A cause that explains the evidence.

03
### Reproduce
The smallest case that fails.

04
### Fix
The cause, not the symptom.

05
### Verify
The reproduction now passes; keep it as a test.

06
### Document
Runbook, incident note, eval case.

ON THE EXAM

No reproduction, no fix. Changing prompts and models at random is swapping parts until the noise stops.

# Common Claude Code issues, and the first check

| SYMPTOM | FIRST CHECK |
| --- | --- |
| MCP server missing | /mcp: pending approval, failed, unset ${VAR} |
| Allowed action still blocked | /status, /permissions: a deny in another scope |
| Context used before you type | /context: long CLAUDE.md, tool search off |
| Prompt to approve a project server | Expected on first use; can be reset |
| Something else is odd | /doctor, then a --debug-file log |

THE TIEBREAKER

Each symptom has a command that shows the cause. Run it before changing anything.

# Production API symptoms, and the first check

| SYMPTOM | FIRST CHECK AND RESPONSE |
|---|---|
| 429 rate_limit_error | Back off, honouring retry-after; smooth bursts; raise limits |
| 529 overloaded_error | Retry with backoff; switch to degraded mode |
| 504 timeout_error | Stream long requests |
| Answers cut off mid-sentence | stop_reason is max_tokens: raise it, or ask for less |
| Cost jumps, traffic flat | Cache reads in usage fell: did the prompt prefix change? |
| Escalating to support | Quote the request-id header |

THE TIEBREAKER

Read the error type, the stop reason and the usage block before touching the prompt.

# Using Claude to debug

- **Give it the evidence:** the reproduction, logs, recent changes.
- **Ask for ranked hypotheses,** each with the evidence for and against.
- **Explore in plan mode** before anything is edited.
- **Have it write the failing test first,** then the fix.
- **A person reviews the fix,** and confirms the cause.

> ON THE EXAM
>
> Claude speeds up each debugging step. It does not let you skip any of them.

> Mereside's IT developers paste code and error messages into a consumer chat app to get help.
>
> Security has found internal hostnames in the pasted text.

## What should Corvane recommend?

A. Ban AI assistance for all developers until a full acceptable-use policy has been written

B. Build an internal chatbot that developers paste code into instead

C. Allow the chat app, but require developers to redact hostnames and secrets from code first

D. Claude Code in their IDE and terminal, under managed and project settings, with repository context

◆ PAUSE AND DECIDE

## What should Corvane recommend?

A Ban AI assistance for all developers until a full acceptable-use policy has been written

*Pushes the habit underground.*

B Build an internal chatbot that developers paste code into instead

*Still a side channel, still pasting.*

C Allow the chat app, but require developers to redact hostnames and secrets from code first

*Relies on every developer, every time.*

D Claude Code in their IDE and terminal, under managed and project settings, with repository context

In the workflow, under controls.

THE SIGNAL

Bring Claude into the workflow, under organisational controls, instead of policing a side channel.

> Tallyhawk wants Claude to review every pull request.
>
> A team lead proposes letting Claude approve and merge any pull request where it finds no issues.

## What should Corvane recommend?

A. Claude comments inline from CI, a human reviewer approves, and branch protection requires that approval

B. Let Claude approve and merge pull requests it finds clean, to save reviewers' time

C. Run Claude's review after merging, so it never slows anyone down

D. Drop human review for small pull requests, and keep it for large ones

◆ PAUSE AND DECIDE

# What should Corvane recommend?

**A** Claude comments inline from CI, a human reviewer approves, and branch protection requires that approval

Findings from Claude, authority with people.

**B** Let Claude approve and merge pull requests it finds clean, to save reviewers' time

*No human accountable for the merge.*

**C** Run Claude's review after merging, so it never slows anyone down

*Findings arrive too late.*

**D** Drop human review for small pull requests, and keep it for large ones

*Small changes cause big incidents too.*

THE SIGNAL

Claude adds findings. A person approves. Branch protection enforces it.

> Tallyhawk support engineers want Claude Code to investigate billing problems directly against the production billing database.

## How should access be set up?

A. Admin database credentials, with auto mode so investigations are not slowed

B. A read-only database user for queries, with writes only through named tools under ask rules, all logged

C. Copy the production database to each engineer's laptop, so investigations never touch production

D. Deny all database access, and have engineers paste query results into Claude by hand

◆ PAUSE AND DECIDE

## How should access be set up?

A. Admin database credentials, with auto mode so investigations are not slowed
*Unbounded writes, unconfirmed.*

B. A read-only database user for queries, with writes only through named tools under ask rules, all logged
Reads by default, confirmed writes, audit.

C. Copy the production database to each engineer's laptop, so investigations never touch production
*Spreads customer data everywhere.*

D. Deny all database access, and have engineers paste query results into Claude by hand
*Back to a side channel.*

THE SIGNAL

Production data: read-only by default, writes confirmed by a person, everything logged.

> Tallyhawk's Claude-assisted import job fails on some customer files, but not others.
>
> An engineer proposes rewriting the prompt and switching to a larger model.

## What should the team do first?

A Rewrite the prompt first, since prompts are the usual cause of failures like these

B Switch to the larger model, and see whether the failures stop

C Form a hypothesis from the failing files, and build a minimal reproduction before changing anything

D Add automatic retries, so any failing files are attempted again until they succeed

◆ PAUSE AND DECIDE

## What should the team do first?

A Rewrite the prompt first, since prompts are the usual cause of failures like these

*A guess, not a diagnosis.*

B Switch to the larger model, and see whether the failures stop

*Swapping parts until the noise stops.*

C Form a hypothesis from the failing files, and build a minimal reproduction before changing anything

Hypothesis and reproduction first.

D Add automatic retries, so any failing files are attempted again until they succeed

*Hides a deterministic failure.*

THE SIGNAL

Reproduce before you fix. A change you cannot test against a reproduction is a guess.

> An engineer clones Tallyhawk's monorepo and starts Claude Code.
>
> The shared help-centre MCP server from the committed .mcp.json does not appear in her tools.

## What should she check first?

A Reinstall Claude Code, since the configuration on her machine may be corrupt

B Copy the server definition into her user settings instead

C Raise a support ticket with Anthropic about the missing server, with the repository name

D Run /mcp or claude mcp list, to see whether it is pending approval, failed to connect, or missing a variable

◆ PAUSE AND DECIDE

## What should she check first?

A Reinstall Claude Code, since the configuration on her machine may be corrupt

*Changes things before diagnosing.*

B Copy the server definition into her user settings instead

*Duplicates config, hides the cause.*

C Raise a support ticket with Anthropic about the missing server, with the repository name

*The cause is almost always local.*

D Run /mcp or claude mcp list, to see whether it is pending approval, failed to connect, or missing a variable

The status command shows the cause.

THE SIGNAL

Run the command that shows the cause before changing anything.

> Tallyhawk plans to give Claude Code to all eighteen engineering teams.

**Which two steps belong before a company-wide rollout? Choose two.**

A Roll out to every team at once, to learn as quickly as possible

B Identify where it adds value, and who keeps human authority over each kind of change

C Set lines of code generated as the main success measure

D Let each team write its own permissions from scratch

E Pilot with one or two teams on a configured baseline, then iterate the configuration

◆ PAUSE AND DECIDE

## Which two steps belong before a company-wide rollout? Choose two.

A Roll out to every team at once, to learn as quickly as possible

*Spreads every mistake to every team.*

B Identify where it adds value, and who keeps human authority over each kind of change

Value and authority come first.

C Set lines of code generated as the main success measure

*More code is not more value.*

D Let each team write its own permissions from scratch

*Eighteen inconsistent policies.*

E Pilot with one or two teams on a configured baseline, then iterate the configuration

Pilot, measure, iterate.

THE SIGNAL

Rollout order: value and authority, baseline, pilot, iterate, then roll out in waves.

> A Tallyhawk engineer runs /context at the start of a session.
>
> A large share of the window is already used before she has typed anything.

**Which two are the most likely causes? Choose two.**

A. A very long CLAUDE.md with whole documents pasted into it

B. The model's context window has shrunk since last week

C. Tool search turned off, so every MCP tool definition loads up front

D. Too many git branches in the repository

E. An API key without the right permissions

◆ PAUSE AND DECIDE

## Which two are the most likely causes? Choose two.

**A** A very long CLAUDE.md with whole documents pasted into it

Loaded every session; move detail into rules or Skills.

**B** The model's context window has shrunk since last week

*Windows do not shrink like that.*

**C** Tool search turned off, so every MCP tool definition loads up front

Re-enable tool search to defer MCP tools.

**D** Too many git branches in the repository

*Branches are not loaded into context.*

**E** An API key without the right permissions

*A key problem fails requests, not context.*

THE SIGNAL

Context used before the first message comes from what loads every session: memory files and tool definitions.

# In five lines

- **In the workflow, not a side channel:** IDE, terminal, CI, under settings.
- **Claude comments, humans approve;** docs reviewed like code.
- **Production read-only by default;** writes confirmed and logged.
- **Rollout:** value and authority, baseline, pilot, iterate, waves, monitor.
- **Debug:** hypothesis, reproduction, fix, verify, document; run the status command first.

> COMING UP
>
> Domain 7 complete. Next: the sequences the exam tests, on one page.